

Course Outline: Storing Information

Title: Storing Information (Anywhere) **Learnpack:** + Almacenando información (en cualquier lugar)
Prerequisite: Working with Files in Python (Week 8, Skill 21) **Target Audience:** Beginner developers who can already build FastAPI endpoints and read/write files in Python, but have never used a database **Total Lessons:** 15 **Estimated Total Length:** ~7,500 words **Format:** Conceptual (0% hands-on)

Course Description

So far in the bootcamp, your applications lose all their data the moment they stop running. You've learned to read and write files, but files aren't designed to handle structured queries, concurrent access, or relationships between data. This course introduces the concept of databases as a general tool and then walks you through TinyDB — a lightweight document database that lets you persist, query, and manage structured data in Python without learning SQL yet.

By the end of this course you will be able to store and retrieve structured data using TinyDB, validate data before saving it with Pydantic, and populate your application with realistic initial data using seeders. These skills prepare you directly for the authentication system you'll build in the next skill.

Course Philosophy

- This course emphasizes:
- **Path of least resistance:** TinyDB gives you real persistence with almost zero setup — no server, no SQL, just Python dictionaries saved to a JSON file.
 - **Validation before storage:** Data that enters your database should always be validated first. Pydantic acts as a gatekeeper so garbage never gets persisted.
 - **Iteration over perfection:** Start with the simplest working persistence, then layer on validation and seeders incrementally.
-

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~450
01 - Why Persist Data?	2	~1,000
02 - TinyDB Basics	2	~1,000
03 - Querying and Modifying	2	~1,000
04 - Validation with Pydantic	3	~1,500
05 - Seeders	2	~1,000
06 - Assessment	2	~1,100

Section	Lessons	Estimated Words
07 - Wrap-Up	1	~450
Total	15	~7,500

Section 00: Welcome

00.0 Welcome 📖 (~450 words)

Learning Objectives:

- Identify what this course covers and why persistence matters at this stage of the bootcamp
- Recognize the connection between file-based storage (previous skill) and database-based storage

Content Outline:

- Quick recap: right now your FastAPI apps store data in memory — restart the server and everything disappears
- The jump from files to databases: files work for simple sequential data, but they become painful when you need to find one record out of thousands, update a single field, or handle multiple operations at once
- What you'll learn: the general concept of databases, hands-on work with TinyDB for persistence, data validation with Pydantic, and seeders for loading initial data
- What you will NOT learn yet: SQL, relational databases, or complex database administration — those come later in the syllabus

Transitions: This lesson sets the stage. Next, you'll explore what a database actually is and why every real application needs one.

Key Principles:

- Persistence means your data survives a server restart
- The simplest tool that solves the problem is the right tool to start with

Examples:

- A to-do app where all tasks vanish every time the server restarts — that's the problem persistence solves
 - Comparing a notebook (file) to a filing cabinet with labeled folders (database) — both store information, but the cabinet lets you find things fast
-

Section 01: Why Persist Data?

01.0 What Is a Database? 📖 (~500 words)

Learning Objectives:

- Define what a database is in general terms
- Distinguish between different categories of databases at a high level (relational, document, key-value)

Content Outline:

- A database is an organized system for storing, retrieving, and managing data — not just a file with content dumped in it
- The key difference: databases provide mechanisms to query, filter, update, and delete specific records without rewriting the entire file
- Brief panorama of database types: relational (PostgreSQL, MySQL — tables with rows and columns), document (TinyDB, MongoDB — collections of JSON-like documents), key-value (Redis — simple pairs for fast lookups)
- Why so many types? Different problems need different tools. A chat application's session cache has different needs than a bank's transaction ledger
- Where TinyDB fits: it's a document database that stores everything as JSON. Perfect for learning persistence concepts without server setup or SQL syntax

Transitions: Now that you know what a database is, the next lesson explains why your application specifically needs one.

Key Principles:

- A database is not just a file — it's a system with built-in tools for structured access
- Different database types exist because different problems have different storage needs

Examples:

- A library: the books are the data, the catalog system is the database — without the catalog, finding a specific book means searching every shelf
- Document database analogy: imagine a drawer full of index cards, each card is a JSON object — you can add cards, search through them by any field, update one card without touching the others

01.1 Why Store Data? 📖 (~500 words)

Learning Objectives:

- Explain why in-memory data is insufficient for real applications
- Identify the moment in an application's lifecycle when persistence becomes necessary

Content Outline:

- The core problem: in-memory variables die with the process. Every time your FastAPI server restarts, all your Python lists and dictionaries reset to empty
- Real-world scenario: imagine building a contacts API — users add 50 contacts, the server restarts, and all contacts are gone. That's not a usable product
- Why not just use files? You already know how to write to CSV and text files from the previous skill. Files work for simple sequential data, but they become painful when you need to find one record out of thousands, update a single field, or handle multiple operations at once
- The database solves: fast lookups by any field, atomic updates to individual records, structured data with consistent shape, and data that survives restarts automatically
- When does persistence kick in? The moment your application needs to remember something between requests or between server restarts, you need a database

Transitions: You understand the "why." Next section, you'll set up TinyDB and start storing actual data.

Key Principles:

- If your data needs to survive a restart, you need persistence
- Files are for sequential storage; databases are for structured access

Examples:

- A shopping cart API: add items during a session, come back tomorrow, and they should still be there — that requires persistence
 - Comparing grep on a 10,000-line CSV to find one user vs. a single TinyDB search query — the database approach is simpler and faster to write
-

Section 02: TinyDB Basics

02.0 Getting Started with TinyDB 📖 (~500 words)

Learning Objectives:

- Install TinyDB in a Python project using pipenv
- Create a TinyDB database instance that persists to a JSON file

Content Outline:

- What TinyDB is: a pure-Python, zero-config document database. No server to install, no connection strings, no admin panel. It just stores Python dictionaries in a local JSON file
- Installation: `pipenv install tinydb`
- Creating your first database: `from tinydb import TinyDB` then `db = TinyDB('db.json')` — that single line creates (or opens) a JSON file that acts as your database
- What happens under the hood: TinyDB serializes Python dictionaries to JSON and writes them to the file. When you read, it deserializes back to dictionaries
- The `.save()` concept: TinyDB auto-saves on every write operation by default. The data hits the file immediately after insert, update, or delete — no manual save needed in the default configuration
- Inspecting your database: open `db.json` in any text editor and you'll see your data as plain JSON — fully readable and debuggable

Transitions: Your database exists. Now let's put data into it.

Key Principles:

- TinyDB requires zero infrastructure — one import and one line of code gives you a working database
- The database file is just JSON — you can always open it and see exactly what's stored

Examples:

- Setting up a `contacts.json` database for a contacts API: `db = TinyDB('contacts.json')` — done
 - Opening the resulting JSON file to see how TinyDB structures the stored documents internally
-

02.1 Insert and Save Records 📖 (~500 words)

Learning Objectives:

- Insert single and multiple records into a TinyDB database
- Understand how TinyDB assigns document IDs automatically

Content Outline:

- Inserting a single record: `db.insert({'name': 'Ana', 'email': 'ana@mail.com'})` — returns a document ID (integer, auto-incremented)
- Inserting multiple records: `db.insert_multiple([{'...'}, {'...'}, {'...'}])` — useful for bulk operations
- Document IDs: every record gets a unique integer ID assigned by TinyDB. You don't choose it — TinyDB manages it. This ID is how you'll reference specific records later
- What gets stored: plain Python dictionaries. TinyDB doesn't enforce any schema — you could insert `{'name': 'Ana'}` and then `{'color': 'blue', 'size': 3}` into the same database. This flexibility is convenient but dangerous without validation (you'll address this in Section 04)
- Verifying insertion: `db.all()` returns a list of all stored documents. Use it to confirm your data is there
- The persistence guarantee: once `insert` returns, the data is already written to the JSON file. Kill the process, restart it, call `db.all()` again — your data is still there

Transitions: You can store data. But storing is only half the job — next you'll learn to find, update, and remove specific records.

Key Principles:

- Every inserted document gets an automatic unique ID
- TinyDB doesn't enforce a schema — discipline must come from your code (Pydantic, covered in Section 04)

Examples:

- Inserting three contacts and then calling `db.all()` to see them listed with their auto-assigned IDs
- Showing what happens when you restart the Python process and call `db.all()` again — the data persists

Section 03: Querying and Modifying

03.0 Search and Filter Data 📖 (~500 words)

Learning Objectives:

- Search for documents in TinyDB using the Query object
- Filter records by field values, including comparisons and combined conditions

Content Outline:

- The Query object: `from tinydb import Query` then `Contact = Query()` — this creates a query builder you'll use to express search conditions
- Basic search: `db.search(Contact.name == 'Ana')` — returns a list of all documents where `name` equals `'Ana'`

- Comparison operators: `==`, `!=`, `<`, `>`, `<=`, `>=` all work as expected on the Query object
- Combining conditions: use `&` for AND and `|` for OR — example: `db.search((Contact.age > 18) & (Contact.city == 'Madrid'))`
- Checking existence: `db.search(Contact.email.exists())` — find all documents that have an `email` field
- Getting one record: `db.get(Contact.name == 'Ana')` — returns the first match or `None`. Useful when you expect exactly one result
- Getting by ID: `db.get(doc_id=3)` — retrieve a specific document by its auto-assigned ID
- Empty results: if no document matches, `search` returns an empty list and `get` returns `None` — always handle this case in your code

Transitions: You can find data. Next, you'll learn to modify and remove records that are already stored.

Key Principles:

- The Query object lets you express conditions in Python syntax — no special query language needed
- Always handle the case where a search returns nothing

Examples:

- Searching for all contacts in a specific city and printing the results
- Using `db.get(doc_id=5)` to retrieve a single contact by ID, then checking if the result is `None` before using it

03.1 Update and Delete Records 📖 (~500 words)

Learning Objectives:

- Update one or more fields in existing TinyDB documents
- Delete documents by query condition or by document ID

Content Outline:

- Updating by query: `db.update({'email': 'new@mail.com'}, Contact.name == 'Ana')` — finds all documents matching the condition and updates the specified fields. Other fields remain untouched
- Updating by ID: `db.update({'email': 'new@mail.com'}, doc_ids=[3])` — update a specific document by its ID
- Deleting by query: `db.remove(Contact.name == 'Ana')` — removes all documents that match
- Deleting by ID: `db.remove(doc_ids=[3])` — removes the specific document
- Clearing everything: `db.truncate()` — empties the entire database. Useful for testing, dangerous in production
- The CRUD summary: you now have Create (`insert`), Read (`search`, `get`, `all`), Update (`update`), and Delete (`remove`). These four operations are the foundation of every database interaction, regardless of the database technology
- Important: updates and deletes are immediate — once the operation returns, the JSON file is already modified. There's no "undo" unless you build one yourself

Transitions: You have full CRUD capabilities. But TinyDB accepts anything — next section introduces Pydantic to make sure only valid data enters your database.

Key Principles:

- CRUD (Create, Read, Update, Delete) is the universal language of data operations — every database supports these four
- Deletes and updates are permanent and immediate — validate before you execute

Examples:

- Updating a contact's email by their name, then verifying the change with a search
 - Accidentally calling `db.truncate()` instead of `db.remove()` — illustrating why careful operation selection matters
-

Section 04: Validation with Pydantic

04.0 Pydantic as a Schema Layer 📖 (~500 words)

Learning Objectives:

- Explain why TinyDB needs an external validation layer
- Use Pydantic models to validate data before inserting it into TinyDB

Content Outline:

- The problem: TinyDB stores plain dictionaries with no schema. You could insert `{'name': 'Ana', 'email': 'ana@mail.com'}` and then `{'x': 42}` — TinyDB won't complain. Over time, your database becomes a mess of inconsistent records
- The solution: Pydantic models act as a gatekeeper. Define the expected shape of your data as a Pydantic class, validate incoming data through it, and only then insert the validated dictionary into TinyDB
- Workflow: define `class Contact(BaseModel): name: str; email: str` → receive raw data → `contact = Contact(**raw_data)` → if valid, `db.insert(contact.model_dump())` → if invalid, Pydantic raises a `ValidationError` before anything touches the database
- Why `.model_dump()`? Pydantic models are not plain dictionaries. TinyDB needs dictionaries, so you convert the validated model to a dict before inserting
- This is the DTO pattern: Pydantic models act as Data Transfer Objects — they define the contract for what data looks like, separate from how it's stored. You already used Pydantic in FastAPI for request/response validation — the same models can guard your database

Transitions: Now that you know the correct approach, the next lesson covers what happens when people skip validation.

Key Principles:

- Never trust incoming data — validate before storing
- Pydantic defines the contract; TinyDB handles the storage. Keep these responsibilities separate

Examples:

- Defining a `Contact` Pydantic model and inserting a validated record into TinyDB

- Trying to create a `Contact` with a missing `email` field — Pydantic raises `ValidationError` and the database is never touched
-

04.1 Storage Anti-Patterns 📖 (~500 words)

Learning Objectives:

- Identify the three most common mistakes when persisting data
- Explain the consequences of each anti-pattern and the correct alternative

Content Outline:

- **Anti-pattern 1: Storing without validation.** Inserting raw user input directly into TinyDB without passing it through Pydantic first. Consequence: inconsistent records — some have `email`, some don't, some have `e_mail` instead. Debugging becomes a nightmare. Correct approach: always validate with Pydantic before `db.insert()`
- **Anti-pattern 2: Storing without structure.** Using different key names for the same concept across records (`{'nombre': 'Ana'}` vs `{'name': 'Juan'}`). Consequence: your queries break because `Contact.name == 'Ana'` won't find records stored under `nombre`. Correct approach: enforce a single Pydantic model so every record has the exact same fields
- **Anti-pattern 3: Querying the same resource multiple times.** Fetching the same record from the database three times in the same request because three different functions each need it. Consequence: unnecessary I/O and slower responses. Correct approach: fetch once, pass the result to the functions that need it
- Why these mistakes are tempting: TinyDB's flexibility makes it easy to "just insert" without thinking. The cost only appears later when you try to query or maintain inconsistent data

Transitions: With validation and awareness of pitfalls in place, the next lesson puts it all together by connecting TinyDB to a FastAPI endpoint.

Key Principles:

- Flexibility without discipline creates chaos — TinyDB's schema-less nature demands that you enforce structure in your code
- Fetch once, use many times — avoid redundant database reads

Examples:

- A database where half the contacts have `email` and the other half have `correo` — showing a search that misses half the records
 - A request handler that calls `db.search(...)` three times for the same data vs. one that calls it once and reuses the result
-

04.2 TinyDB in a FastAPI Endpoint 📖 (~500 words)

Learning Objectives:

- Connect TinyDB persistence to a FastAPI endpoint using Pydantic validation
- Describe the complete flow from HTTP request to database storage to HTTP response

Content Outline:

- The full picture: a POST endpoint receives JSON → FastAPI validates it with a Pydantic model (you already know this) → the validated data is inserted into TinyDB → the endpoint returns the stored record with its assigned ID
- Code walkthrough: defining the Pydantic model, initializing the TinyDB instance, creating a POST endpoint that calls `db.insert(contact.model_dump())`, and returning the result
- GET endpoint: creating a GET endpoint that calls `db.all()` or `db.search(...)` and returns the results as JSON
- Where to initialize the database: `db = TinyDB('db.json')` at the module level so all endpoints share the same instance
- What this replaces: up until now your FastAPI endpoints stored data in a plain Python list that reset on restart. Now the data survives
- Testing the persistence: start the server, POST a contact, stop the server, start it again, GET contacts — the data is still there

Transitions: Your API now persists data. But an empty database isn't useful for development or testing — next section covers seeders to load initial data automatically.

Key Principles:

- The persistence layer sits behind your API — the frontend doesn't know or care whether you use TinyDB, PostgreSQL, or anything else
- Initialize the database once and share the instance across endpoints

Examples:

- A minimal FastAPI app with one POST and one GET endpoint backed by TinyDB, showing the complete request-to-storage flow
- Stopping and restarting the server to prove that data persists between sessions

Section 05: Seeders

05.0 What Is a Seeder? 📖 (~500 words)

Learning Objectives:

- Define what a seeder is and explain its purpose in application development
- Identify when and why you need seed data

Content Outline:

- A seeder is a script that populates your database with initial data — realistic, structured records that your application needs to function or that you need for testing and development
- Why seed data matters: an empty database is useless for development. You can't test a contacts page if there are no contacts. You can't demo your API if every GET returns an empty list. Seeders give you a consistent starting point
- What goes in a seeder: realistic but fake data. Names, emails, dates — data that looks real but doesn't belong to actual people. The goal is to simulate a production-like environment

- When to run seeders: at the beginning of development, after resetting the database, when onboarding a new team member, or when setting up a testing environment
- Seeders are not migrations: a seeder fills the database with data. A migration changes the database structure. You'll learn about migrations later when working with SQL databases and ORMs
- Idempotency consideration: a good seeder should check whether data already exists before inserting, or clear the table first with `db.truncate()`, to avoid duplicate records on repeated runs

Transitions: You know what seeders are. Next, you'll build one and learn to run it as a command.

Key Principles:

- A seeder gives your application a predictable starting state — essential for development and testing
- Seed data should be realistic enough to test real scenarios but never contain actual personal data

Examples:

- Starting a contacts API with zero data vs. running a seeder that creates 10 sample contacts — the difference in developer experience when testing
 - A team of three developers, each with an empty database, running the same seeder to get the same starting data
-

05.1 Running Seeders with pipenv 📖 (~500 words)

Learning Objectives:

- Create a seeder script that populates a TinyDB database with initial data
- Configure and run the seeder as a pipenv script command

Content Outline:

- Structure of a seeder script: a Python file (e.g., `seed.py`) that imports TinyDB, connects to the database, optionally clears existing data with `db.truncate()`, defines a list of seed records, and inserts them with `db.insert_multiple()`
- Using Pydantic in the seeder: validate your seed data through the same Pydantic models your API uses. If the seed data doesn't pass validation, you have a bug in your seed data — better to catch it here than in production
- Configuring pipenv scripts: in your `Pipfile`, under `[scripts]`, add `seed = "python seed.py"`. Now you can run `pipenv run seed` from the terminal
- Why a pipenv script? It runs the command inside the virtual environment automatically, ensuring the correct Python version and dependencies are used. It's also self-documenting — any developer can see `pipenv run seed` in the `Pipfile` and know how to populate the database
- The workflow in practice: clone the repo → `pipenv install` → `pipenv run seed` → `pipenv run dev` — the database starts populated and the API is immediately usable
- Tip: print a summary at the end of the seeder (e.g., "Seeded 10 contacts successfully") so you get confirmation it ran

Transitions: You can now persist data, validate it, and load initial records. Time to test everything you've learned.

Key Principles:

- Seeders should be runnable with a single command — `pipenv run seed` is the standard
- Validate seed data through the same models your application uses — consistency is non-negotiable

Examples:

- A complete `seed.py` file that creates 5 contacts using Pydantic validation and inserts them into TinyDB
 - The `[scripts]` section of a `Pipfile` with the seed command configured
-

Section 06: Assessment

06.0 Persistence Patterns Recap 📖 (~500 words)

Learning Objectives:

- Summarize the complete data persistence workflow: validate → store → query → seed
- Distinguish between correct patterns and anti-patterns covered in this course

Content Outline:

- Review of the full persistence flow: raw data → Pydantic validation → `model_dump()` → `db.insert()` → stored in JSON file → queryable with `db.search()`, `db.get()`, `db.all()`
- The CRUD mapping: Create = `insert`, Read = `search/get/all`, Update = `update`, Delete = `remove`
- Validation as gatekeeper: Pydantic catches invalid data before it reaches TinyDB. This pattern applies to any database, not just TinyDB
- Seeders as development infrastructure: a single `pipenv run seed` command gives every developer the same starting data
- Quick anti-pattern review: storing without validation, inconsistent field names, redundant queries for the same data
- Looking ahead: TinyDB is great for learning and small projects, but it's not designed for production traffic, concurrent users, or complex relationships. SQL databases (coming in Week 10) solve those problems — but the concepts you learned here (CRUD, validation, seeders) transfer directly

Transitions: With the full picture in mind, the next lesson tests your understanding.

Key Principles:

- The persistence pattern is universal: validate → store → query. The specific tool changes, but the workflow stays the same
- Everything you learned here — CRUD, validation, seeders — applies to every database you'll ever use

Examples:

- A side-by-side comparison of the persistence workflow with TinyDB vs. the same workflow conceptually with a SQL database — showing the parallel
 - The journey of a single contact record from HTTP request to validated Pydantic model to TinyDB document to API response
-

06.1 Knowledge Check 🧠 (~700 words)

Learning Objectives:

- Demonstrate understanding of database concepts, TinyDB operations, Pydantic validation, and seeders
- Identify correct and incorrect persistence practices

Question Format: Multiple choice (5 questions)

Questions:

Q1. Your FastAPI app stores contacts in a Python list. After restarting the server, all contacts disappear. What is the root cause?

- A) The list was declared inside a function instead of at module level
- B) In-memory data does not survive process restarts — you need a persistence layer
- C) FastAPI clears variables on restart as a security measure
- D) The list exceeded Python's memory limit and was garbage collected

Correct: B

Q2. You execute `db.insert({'name': 'Ana', 'email': 'ana@mail.com'})` in TinyDB. What happens immediately after this line runs?

- A) The data is held in memory and written to the file only when you call `db.save()`
- B) The data is validated against a schema before being stored
- C) The record is written to the JSON file and assigned an auto-incremented document ID
- D) TinyDB checks for duplicates and rejects the insert if a record with the same name exists

Correct: C

Q3. A developer inserts records into TinyDB without using Pydantic. After a week, some records have `email`, others have `correo`, and a few have no email field at all. What anti-pattern does this illustrate?

- A) Querying the same resource multiple times
- B) Using TinyDB instead of a relational database
- C) Storing data without validation or enforced structure
- D) Forgetting to call `db.save()` after each insert

Correct: C

Q4. What is the purpose of a seeder in application development?

- A) To encrypt sensitive data before it enters the database
- B) To change the structure of database tables when requirements evolve
- C) To populate the database with initial data for development, testing, or demos
- D) To back up the database to a remote server on a scheduled basis

Correct: C

Q5. A Pydantic model `Contact` requires a `name` (str) and `email` (str). A developer tries to insert `{'name': 'Ana'}` directly into TinyDB without passing it through the Pydantic model. What happens?

- A) TinyDB rejects the record because the `email` field is missing
- B) The record is inserted successfully because TinyDB has no schema enforcement — the missing field goes unnoticed
- C) Python raises a `KeyError` when accessing the `email` field during insertion
- D) Pydantic automatically intercepts the insert and raises a `ValidationError`

Correct: B

Evaluation Criteria:

- 5/5: Excellent — solid understanding of persistence concepts, TinyDB operations, and validation patterns
 - 4/5: Good — minor gaps, review the missed topic
 - 3/5: Acceptable — revisit Sections 02–04 for reinforcement
 - Below 3: Review the full course before proceeding
-

Section 07: Wrap-Up

07.0 What Comes Next (~450 words)

Content Outline:

- Course recap: you went from in-memory data that vanished on restart to a persistent, validated, seeded database. You now understand what a database is, how to perform CRUD operations with TinyDB, how to guard your data with Pydantic validation, and how to set up seeders for consistent development environments
- Connection to the bigger picture: every real application needs persistence. The pattern you learned — validate, store, query — is the same whether you're using TinyDB, PostgreSQL, MongoDB, or any other database. The tools change; the thinking doesn't
- What comes next: in the very next skill you'll build an authentication system. You'll store users with hashed passwords in your database and manage sessions with JWT tokens. Everything you learned here — especially Pydantic validation and TinyDB CRUD — will be the foundation for that work
- Further down the road: in Week 10 you'll learn SQL and PostgreSQL, and in Week 11 you'll use an ORM (SQLModel) to connect Python classes directly to database tables. TinyDB was your first step into persistence; SQL databases are the next level
- Resources: TinyDB official documentation (tinydb.readthedocs.io), Pydantic documentation (docs.pydantic.dev), the Pipfile scripts documentation for custom commands
- Final encouragement: your applications now have memory. That's a fundamental shift — from temporary scripts to real software that remembers. Every feature you build from here on can rely on persistent data, and that changes everything

Note: This is conclusion only — no theory, exercises, or assessment.